

Resumen: *Code as Design* — Los tres ensayos de Jack W. Reeves

Tomás Felipe Melli

June 19, 2026

Índice

1	Marco general de los tres ensayos	2
2	Ensayo 1 — “What Is Software Design?” (1992)	2
2.1	Tesis central	2
2.2	Las consecuencias de “el código es el diseño”	2
2.3	Hardware vs. Software: por qué no debemos copiar al hardware	3
2.4	Validación: testing y debugging <i>son</i> diseño	3
2.5	Crítica al modelo de proceso tradicional	3
2.6	Todo es diseño, y todo interactúa	4
2.7	Notaciones de diseño y el rol de C++	4
2.8	Documentación auxiliar	5
2.9	Resumen final del autor (14 puntos)	5
3	Ensayo 2 — “What Is Software Design: 13 Years Later” (2005)	6
3.1	Crítica A — “Si el código es el diseño, los programadores serían diseñadores; pero no lo son, luego el código no es el diseño”	6
3.2	Crítica B — “Decir que el código es el diseño es decir ‘no diseñes, solo codificá’ ”	6
3.3	Crítica B’ — “¿Y el programador menos capaz?” (<i>Less Able Programmer</i>)	6
3.4	Crítica C — “La meta de la ingeniería es un <i>producto</i> , no documentación”	6
3.5	Crítica D — “El código fuente es demasiado alto nivel; es una <i>especificación</i> , el diseño real es lo que sale del compilador”	7
4	Ensayo 3 — “Letter to the Editor” (carta original, 1992)	7
4.1	La analogía del puente (replantada)	7
4.2	La economía como explicación de todo	7
4.3	“Diseñar lo es todo”: codificar es diseñar	7
4.4	Aclaración clave y las dos “correcciones de percepción”	8
4.5	Contra las notaciones “independientes del lenguaje”	8
4.6	Dos problemas que él mismo admite del “código como diseño”	8
4.7	Mirada sobre los procesos de desarrollo	8
4.8	La “fantasía colectiva” y la conclusión	8
5	Síntesis comparada de los tres ensayos	9

1 Marco general de los tres ensayos

Los tres textos de Jack W. Reeves (recopilados como “*Code as Design: Three Essays by Jack W. Reeves*”, de *developer.* Magazine*) giran en torno a una sola tesis central: **programar es fundamentalmente una actividad de diseño**, y la única representación final y verdadera del diseño de un sistema es el **código fuente mismo**.

Los tres textos, en orden de lectura del apunte:

1. “**What Is Software Design?**” (1992): el ensayo original, publicado en el ya desaparecido *C++ Journal*. Tras un período de oscuridad, ganó difusión gracias a la web y al libro de Robert Martin *Agile Software Development: Principles, Patterns, and Practices*.
2. “**What Is Software Design: 13 Years Later**” (2005): Reeves responde a las críticas más comunes que recibió.
3. “**Letter to the Editor**” (1992): la carta original que dio origen a las ideas del primer ensayo (cronológicamente, es el punto de partida de todo).

Aunque las ideas siguen siendo frescas y controvertidas, con los años se volvieron conceptos clave detrás de corrientes como **Agile**, las metodologías basadas en *craft* (oficio), el **test-first design**, el **test-driven development (TDD)** y el **refactoring**.

El hilo conductor de los tres

La distinción que ordena todo: **diseñar software** (el *proceso*) $\neq$ **un diseño de software** (el *producto*, que es el código). Y el criterio operativo: un documento es “de diseño” cuando es lo bastante detallado y **no ambiguo** como para construirse **mecánicamente**; en software ese documento es el **listado de código fuente**.

2 Ensayo 1 — “What Is Software Design?” (1992)

2.1 Tesis central

La programación no se trata de *construir* software; se trata de *diseñar* software.

Reeves sostiene que la industria pasó casi 10 años (al momento de escribir) sin captar una distinción sutil pero profunda: la diferencia entre **el proceso de hacer un diseño de software** y **lo que un diseño de software realmente es**.

Conclusión: no somos verdaderos ingenieros de software porque no entendemos qué es realmente un diseño de software. La industria construyó **falsos paralelos con la ingeniería de hardware** e ignoró otros paralelos válidos.

El argumento clave: ¿cuál es el documento de diseño en software?

El objetivo final de toda actividad de ingeniería es producir algún tipo de **documentación**. Cuando el diseño se termina, la documentación se entrega al **equipo de manufactura** (un grupo distinto, con habilidades distintas), que puede construir el producto —y muchas copias— sin más intervención de los diseñadores.

¿Qué documento del ciclo de vida del software cumple ese criterio? Respuesta de Reeves:

El único documento que satisface el criterio de un diseño de ingeniería es **el listado de código fuente**.

El ensayo **no intenta probar** que el código fuente es el diseño; lo **asume** y examina las consecuencias, mostrando que explica mejor muchos hechos observados de la industria (incluida la popularidad de C++).

2.2 Las consecuencias de “el código es el diseño”

Consecuencia 1: el software es baratísimo de *construir*

Si el código fuente es el diseño, entonces **la construcción del software la hacen los compiladores y los linkers** (lo que llamamos “*hacer un build*”).

- La inversión de capital para construir software es mínima: una computadora, un editor, un compilador y un linker.
- Hacer un build cuesta solo un poco de tiempo.
- Comparación: compilar 50.000 líneas de C++ puede parecer eterno, **pero ¿cuánto tardaría construir un sistema de hardware con una complejidad de diseño equivalente?**

Es un **punto ciego** para la mayoría de las organizaciones: el software es **casi gratis** de construir.

Consecuencia 2: un diseño de software es relativamente fácil de *crear*

Escribir (es decir, *diseñar*) un módulo típico de 50 a 100 líneas suele ser un par de días (depurarlo por completo es otra historia). Resultado: **los diseños de software se vuelven enormes y complejos muy rápido:**

- Proyectos escolares: varios miles de líneas.
- Productos comerciales típicos: cientos de miles de líneas.
- Muchos diseños llegan a millones de líneas, y **evolucionan constantemente**.

2.3 Hardware vs. Software: por qué no debemos copiar al hardware

- El hardware complejo **tampoco está libre de errores**: microprocesadores con fallos lógicos, puentes caídos, represas rotas, aviones caídos, recalls de autos — todos por **errores de diseño**.
- El hardware complejo tiene **fases de construcción caras**, lo que limita cuántas empresas pueden producirlo. **El software no tiene esa limitación**: miles de organizaciones y sistemas cada vez más complejos.

Conclusión: la industria del software **no** resolverá sus problemas imitando al hardware. Si acaso, con CAD/CAM, **la ingeniería de hardware se está pareciendo cada vez más al desarrollo de software**.

Diseñar software es, en esencia, **un ejercicio de gestión de la complejidad**, que existe en tres niveles: (1) dentro del propio diseño; (2) dentro de la organización; (3) dentro de la industria. El software se parece más a **sistemas sociales u orgánicos complejos** que al hardware: las especificaciones son fluidas y los equipos cambian a mitad de camino.

2.4 Validación: testing y debugging *son* diseño

Los ingenieros de hardware validan y refinan sus diseños **antes** de construir (análisis estructural, modelos por computadora, simulaciones, modelos a escala, túneles de viento, prototipos de vuelo). El software también valida, solo que no lo llamamos ingeniería:

En software, validar y refinar el diseño se llama **testing y debugging**.

Revelación 2: es más barato construir y probar que cualquier otra cosa

Como los builds no cuestan casi nada, es más barato construir el diseño y probarlo que demostrarlo formalmente.

- El testing **no solo busca corregir**: es parte de **refinar** el diseño.
- Aunque las demostraciones formales fueran tan automáticas como un compilador, **igual haríamos ciclos build/test**. Por eso las pruebas formales nunca tuvieron mucho interés práctico.

La realidad: diseños cada vez más complejos, hechos por más gente, codificados en algún lenguaje y **validados/refinados vía build/test**. El proceso es propenso a errores y poco riguroso, y se agrava porque muchos desarrolladores **se niegan a creer** que funciona así.

2.5 Crítica al modelo de proceso tradicional

Los procesos tradicionales intentan **segregar las fases** en compartimentos estancos: el alto nivel se “congela” antes de codificar; los programadores son “obreros”; el testing solo corrige errores de construcción. Reeves: es un **modelo incorrecto**.

- Ninguna otra industria toleraría una **tasa de retrabajo de más del 100%** en manufactura. En software hasta el código más chico se reescribe durante el testing. Eso es propio de un **proceso creativo de diseño**, no de manufactura.
- Nadie espera un diseño perfecto a la primera; igual debe pasar por refinamiento (aunque sea para demostrar que es perfecto).
- Lección del management japonés: **es contraproducente culpar a los trabajadores por errores del proceso**. Hay que **revisar el proceso**, no forzar al software a un modelo equivocado.

Prueba de fuego de la “ingeniería de software”

La ingeniería se trata de **cómo se hace el proceso**, no de si el documento final necesita un sistema CAD para producirse.

2.6 Todo es diseño, y todo interactúa

- Codificar es diseño; testing y debugging son diseño; lo que llamamos “diseño de software” también lo es.
- El software es **barato de construir pero carísimo de diseñar**. Hay muchos aspectos y vistas, y **todos se interrelacionan**: los aspectos de una capa se entrometen en las otras.
- **No hay jerarquía de importancia**: un error en el módulo más bajo puede ser tan fatal como uno en el nivel más alto. El diseño debe ser **completo y correcto en todos sus aspectos**, o todos los builds serán erróneos.

Por qué el diseño de alto nivel no puede “congelarse”

- El diseño **no está completo hasta que se codifica y se prueba**.
- El alto nivel **no es un diseño completo**: es un **andamiaje** para el diseño detallado.
- Tenemos capacidad muy limitada de validar rigurosamente el alto nivel.
- El diseño detallado **debería influir** en el de alto nivel.
- Si se “congela” algún aspecto fuera del refinamiento, no sorprende que el resultado sea pobre o inviable.

El software depende de demasiadas cosas (hardware que no se comporta como se creía, una rutina de biblioteca con una restricción no documentada...). Esos problemas aparecen en el testing porque **no había forma de descubrirlos antes, y fuerzan cambios** que se propagan (Ley de Murphy). Los gerentes lo perciben como “*hacking*”, pero es la **realidad del desarrollo**.

Craft vs. Engineering

La experiencia nos lleva en la dirección correcta (eso es *craft*/oficio), pero solo hasta cierto punto en territorio desconocido. A partir de ahí, **mejorar lo que tenemos mediante un proceso controlado de refinamiento es ingeniería**.

2.7 Notaciones de diseño y el rol de C++

- Escribir los documentos de diseño **después** del código produce documentos mucho más precisos: solo el diseño final, reflejado en el código, es el que se refina en el build/test. La probabilidad de que el diseño inicial quede sin cambios es **inversamente proporcional** a la cantidad de módulos y programadores (tiende a cero).
- Necesitamos buen diseño en **todos los niveles**; usar **lo que ayude** (structure charts, diagramas de Booch, tablas de estado, PDL...).
- **PERO** esas notaciones **no son un diseño de software**. Tarde o temprano hay que crear el diseño real, en algún lenguaje de programación.

El problema de la traducción y por qué C++

- No hay una notación igualmente apta para alto nivel y diseño detallado; las notaciones de alto nivel **deben traducirse** al lenguaje destino, y esa traducción **lleva tiempo e introduce errores**.
- Conviene que **los diseñadores originales escriban el código original**, en vez de que otro traduzca un diseño independiente del lenguaje.
- Se necesita **una notación unificada para todos los niveles**: un lenguaje que también sirva para capturar conceptos de alto nivel. **Ahí entra C++**.

Por qué C++ se volvió popular (según Reeves): hace más fácil **diseñar software y programar al mismo tiempo**; es apto para proyectos reales y **más expresivo como lenguaje de diseño** (expresa info de alto nivel sobre los componentes, facilita refinar, y su chequeo de tipos más fuerte ayuda a detectar errores).

La evidencia histórica

Ganaron popularidad las mejoras en **técnicas y lenguajes de programación reales** (programación estructurada → Pascal; OOD → C++). **No** funcionaron de forma universal: herramientas CASE, structure charts, diagramas de Warner-Orr, de Booch, de objetos — su debilidad fundamental es que **no son un diseño de software**. La única notación de diseño verdaderamente extendida es **PDL** (que se parece al código). Lo mismo con el proceso: del *waterfall* se pasó al **espiral y el prototipado rápido**, que en el fondo son **excusas para empezar a codificar antes**.

2.8 Documentación auxiliar

Los documentos de diseño (el código) no son los únicos necesarios. Dos necesidades que requieren documentación auxiliar:

1. **Capturar información del espacio del problema** que no llegó directamente al diseño (entendimiento que ayudó a decidir y conviene guardar por si hay que cambiar el modelo).
2. **Documentar aspectos del diseño difíciles de extraer del código** (muchos se representan mejor de forma **gráfica**, difíciles de poner como comentarios).

Puntos importantes:

- No es un argumento a favor de una notación gráfica en lugar del lenguaje (es análogo a las descripciones textuales que acompañan los planos en hardware).
- **El código fuente determina cuál es el diseño real, no la documentación auxiliar.**
- Lo ideal: herramientas que **generen la documentación auxiliar a partir del código**.
- Mantenerla **al mínimo y lo más informal posible** hasta lo más tarde posible.

2.9 Resumen final del autor (14 puntos)

1. El **software real** corre en computadoras: unos y ceros en un medio magnético. **No** es un listado en C++ ni en ningún lenguaje.
2. Un **listado de programa es un documento que representa un diseño**. Compiladores y linkers **construyen** los diseños.
3. El software real es **increíblemente barato de construir**.
4. El software real es **increíblemente caro de diseñar** (muy complejo; casi todos los pasos son diseño).
5. **Programar es una actividad de diseño**: un buen proceso lo reconoce y no duda en codificar cuando tiene sentido.
6. **Codificar tiene sentido más a menudo de lo que se cree**: plasmar el diseño en código revela omisiones; cuanto antes, mejor.
7. Como el software es tan barato de construir, los **métodos formales de validación** no sirven mucho: es más fácil construir y probar.
8. **Testing y debugging son actividades de diseño** (validación y refinamiento). Un buen proceso no los recorta.
9. **Hay otras actividades de diseño** (alto nivel, módulos, estructural, arquitectónico): un buen proceso las incluye.
10. **Todas las actividades de diseño interactúan**: un buen proceso permite que el diseño cambie, a veces radicalmente.
11. **Muchas notaciones son útiles** como documentación auxiliar, pero **no son** un diseño de software.
12. El desarrollo de software es **todavía más un oficio (craft) que una ingeniería**, por la falta de rigor al validar y mejorar el diseño.
13. Los avances reales dependen de **avances en técnicas y lenguajes de programación**. C++ es uno de esos avances.
14. C++ es un paso en la dirección correcta, **pero hacen falta más avances**.

Ideas clave del ensayo 1 (en una línea)

Tesis: el código fuente *es* el diseño; programar *es* diseñar. **Las dos “revelaciones”:** (1) construir software es casi gratis; (2) es más barato construir y testear que validar formalmente. **Manufactura del software** = compilar y linkear (lo hacen las máquinas). **Testing y debugging** = validación y refinamiento del diseño. **Las notaciones** (UML, Booch, CASE) no son el diseño. El software se parece a **sistemas orgánicos/sociales**, no al hardware.

3 Ensayo 2 — “What Is Software Design: 13 Years Later” (2005)

Trece años después, Reeves **no cambia su tesis**; la defiende respondiendo a las críticas más comunes. Todo gira alrededor de **una distinción que, según él, la gente se niega a entender**:

Diseño como *proceso* $\neq$ Diseño como *producto*.

Es la diferencia entre *escribir* un trabajo (el proceso) y *el trabajo* en sí (el producto). El ensayo está organizado como una réplica a cinco críticas (A, B, B', C, D).

3.1 Crítica A — “Si el código es el diseño, los programadores serían diseñadores; pero no lo son, luego el código no es el diseño”

Reeves lo desarma como una **falacia de petición de principio** (*begging the question*): el argumento **parte de asumir** que programar es *manufactura* (el programador como operario de línea) y, desde esa premisa, concluye que el código no puede ser diseño. Es circular.

Aclaración de método: él **no pretende demostrar** que “el código es el diseño”. Qué es un “diseño” es, en parte, una cuestión de **definición**. Lo que afirma es que **asumir** que el código es el diseño **explica mejor** los hechos observados que cualquier supuesto alternativo.

3.2 Crítica B — “Decir que el código es el diseño es decir ‘no diseñes, solo codificá’ ”

La que más lo irrita, porque **nunca dijo eso**. Vuelve a la distinción proceso/producto:

- **Necesitamos buen diseño en todos los niveles**: buenas **arquitecturas** (alto nivel), buenas **abstracciones** (diseño de clases) y buenas **implementaciones** (bajo nivel). Usar UML o tarjetas CRC para explorar alternativas está perfectamente bien.
- **PERO** esas herramientas **no son** el diseño de software: el diseño real termina en un lenguaje de programación.
- Lo fundamental: **el proceso no está completo hasta que se escribió y se probó el código**.

Sobre *cómo* pensar el diseño, su postura es **liberal**: alguien con los pies sobre el escritorio mirando el techo puede estar “diseñando” tan en serio como quien juega con UML. **Lo que importa es el código resultante**, no el método:

- Si el código es bueno, ¿importa cómo se llegó a él?
- Si el código es malo, ¿importa cuánta documentación se hizo antes de escribirlo mal?

Rechaza el “**punto medio**” de cantidad de diseño: *no existe* esa dosis exacta. **No hay bala de plata ni “manera correcta” de diseñar**. A veces una hora de pensar antes ahorra mucho; otras, 5 minutos de testing revelan algo impensado. La única validación real es **construir y probar**.

Cierre: **el código fuente no es la única documentación necesaria** (la auxiliar también importa); es el **documento maestro** de diseño, pero rara vez el único.

3.3 Crítica B' — “¿Y el programador menos capaz?” (*Less Able Programmer*)

La idea criticada: como solo los mejores pueden “diseñar y codificar” a la vez, hay que imponer pasos y productos intermedios para compensar al programador promedio.

Reeves lo compara con “¿qué hacemos con el médico menos capaz?”. La medicina igual **exige altos estándares** de inteligencia, formación y experiencia. En software, esta crítica equivale a **querer sustituir inteligencia, aptitud y experiencia por proceso**. No hay evidencia de que llenar de diagramas y revisiones haga que alguien termine entendiendo y codificando bien. Además, **usar bien UML ya requiere su propia experiencia**.

3.4 Crítica C — “La meta de la ingeniería es un *producto*, no documentación”

Algunos dicen que los ingenieros “construyen cosas” y esas cosas son tan producto de la ingeniería como cualquier documento. Reeves: eso **esquiva la pregunta**. Admite que hay ingenieros que construyen con poca o ninguna documentación formal, pero esos suelen ser **productos únicos (one-off), hechos por individuos**.

En cuanto la ingeniería **involucra a más de un par de personas o tiene una fase formal de manufactura**, la documentación pasa a ser cada vez más **el verdadero producto** del esfuerzo (Toyota, Motorola, Boeing, Lockheed sí producen documentación). La pregunta filosófica: cualquiera que se llame ingeniero **sabe cómo luce un documento de diseño** en su campo. ¿Pueden decir lo mismo los “ingenieros de software”?

3.5 Crítica D — “El código fuente es demasiado alto nivel; es una *especificación*, el diseño real es lo que sale del compilador”

Reeves apela a la definición estándar:

- Especificación = el *qué*.
- Diseño = el *cómo*.

El compilador tiene cierta flexibilidad para decidir el *cómo* del código objeto, pero **no hay creatividad** en ello. La línea:

Cuando un documento es lo bastante detallado, completo y **no ambiguo** como para interpretarse **mecánicamente** (por una computadora o por un operario), es un **documento de diseño**. Si todavía requiere **interpretación humana creativa**, no lo es.

En desarrollo de software, ese documento de diseño es el **listado de código fuente**.

Ideas clave del ensayo 2 (para fijar)

Distinción que ordena todo: diseño como **proceso** vs. diseño como **producto**. No pretende *probar* “el código es el diseño”: lo **asume** porque **explica mejor** los hechos. “El código es el diseño” **no** significa “no diseñes”. **No existe** la dosis exacta ni la “manera correcta”: la única validación es **build/test**. No se puede **sustituir experiencia por proceso**. **Especificación = qué / Diseño = cómo** interpretable mecánicamente, sin creatividad → el diseño es el **listado de código fuente**.

4 Ensayo 3 — “Letter to the Editor” (carta original, 1992)

Cronológicamente es el **origen** de todo: la carta al editor del *C++ Journal* que luego se transformó en el primer ensayo. El propio autor la considera **mejor escrita que el artículo**: más personal y apasionada. Las ideas son las mismas, pero acá brillan tres recursos conceptuales.

4.1 La analogía del puente (replantada)

El editor había comparado: “*puede no ser un buen puente, pero un puente es*” (sustituyendo “puente” por “diseño de software”), insinuando: ¿confiarías en algo construido con poco o ningún diseño? Reeves muestra que **la comparación está mal armada** y la reordena:

“*Puede no ser un buen **diseño** de puente, pero un **diseño** de puente es.*”

Ante “¿cruzarías primero ese puente?”, la respuesta válida no es “no, un mal diseño es peor que nada”, sino “**¿cuál puente?**”. **El diseño no es el puente**. No se construyen puentes a partir de diseños en bruto: antes se refinan mucho (análisis, modelos por computadora, simulaciones, modelos a escala, túnel de viento). **A eso lo llamamos ingeniería** (y ni así sale siempre perfecto: el chiste del puente de Tacoma). En software hacemos lo mismo, solo que **lo llamamos “testing y debugging”**.

4.2 La economía como explicación de todo

La pregunta recurrente —¿por qué los desarrolladores no “ingenierizan” mejor sus diseños?— tiene, para Reeves, una respuesta que casi nadie da: **economía simple**. El software es **baratísimo de construir**. Compilar y linkear 50.000 líneas es trivial comparado con ensamblar una placa de 50.000 componentes o un puente de 50.000 elementos. Por eso **no hacemos pruebas formales de corrección**: es más barato construir y probar.

Crítica: como **no reconocemos testing y debugging como parte del diseño**, los tratamos como mero “control de calidad” y les dedicamos lo mínimo, aunque se lleven la mitad del ciclo de vida. Nota de humildad comparativa: otras ingenierías probablemente tampoco sepan qué porcentaje de su tiempo es “diseñar” vs. “probar y corregir”; algunas son mejores que el software, **otras seguramente peores** (lo que cuesta “diseñar” un avión nuevo).

4.3 “Diseñar lo es todo”: codificar es diseñar

La frase más memorable:

Decir que los programadores no deberían tener que diseñar es como decir que **los peces no deberían tener que nadar**.

Cuando programa, está **creando un diseño desde la nada**: elige estructuras de datos y algoritmos, evalúa alternativas según ventajas/desventajas, y descompone en submódulos cuando el diseño se vuelve demasiado complejo. Después prueba y refina —y el refinamiento viene tanto de **encontrar errores** como de **walkthroughs entre pares y revisiones formales**. Regla de oro: **el diseño debe ser correcto, o todo lo construido a partir de él será erróneo**.

También: su módulo es **una parte pequeña** de un sistema grande; no puede atender a la vez cientos de módulos y miles de detalles. Y existen “**otros aspectos**” que no caen limpiamente en “estructuras de datos y algoritmos” —justamente **lo que la mayoría llama “diseño de software”**. Alto y bajo nivel **se afectan mutuamente**; hay que refinar todos los aspectos durante todo el ciclo y **no “congelar” ninguno**.

4.4 Aclaración clave y las dos “correcciones de percepción”

Sus colegas interpretaron sus arengas como “*Jack dice: olvidate del diseño y empezá a codificar*”. **Falso**. No está en contra del diseño tradicional; pide **dos cambios de percepción**:

1. Reconocer que **los resultados de las etapas tempranas de diseño no son un diseño completo** (igual que los primeros bocetos no son un diseño completo de puente).
2. **Capturar el pensamiento de diseño en una notación que sea un verdadero esqueleto de diseño de software** — es decir, **un lenguaje de programación**.

Imagen: a la computadora **no le importa cómo llegaste** al diseño final, igual que al obrero que construye el puente no le importa cómo se refinó y validó. Lo único que importa para *construir* es que el diseño sea suficiente. Pero **cómo crear un buen diseño sí nos importa muchísimo a los que lo creamos**: mejor diseño temprano = menos refinamiento después.

4.5 Contra las notaciones “independientes del lenguaje”

El diseño de software es **traducir conceptos del espacio del problema a un lenguaje de programación**. Esa traducción la hacen **humanos**, los lenguajes suelen ser inadecuados para expresar el problema directamente, y **toda traducción pierde información** —peor aún con varias traducciones y **distinta gente en cada paso**. Conclusión: no tiene sentido **agregar pasos de traducción** evitables. No hay nada sagrado en C++ (ni Ada, C, Smalltalk, LISP); ningún lenguaje es el “nativo” de la computadora. **Los lenguajes de programación ya son, en sí mismos, una notación de diseño**.

4.6 Dos problemas que él mismo admite del “código como diseño”

1. Hasta los mejores lenguajes son **débiles para expresar ciertos aspectos**: la información *está* en el código, pero **cuesta sacarla en forma legible para humanos** (los “otros aspectos”).
2. Hay **información del espacio del problema** que entró en el diseño pero **no puede reconstruirse** desde el código; conviene guardarla por si hay que cambiar el diseño. El **comentario típico no alcanza**.

→ Por eso la **documentación auxiliar** es tan importante como en cualquier ingeniería, **pero no hay que confundirla con el diseño**. La salida real al problema 1 sería tener **lenguajes más expresivos** (otra vez, C++ como avance).

4.7 Mirada sobre los procesos de desarrollo

- **Waterfall / MIL-STD**: no dejan escribir una línea hasta producir y revisar “toneladas” de documentación; encima, quienes la escriben suelen irse, y gente **más joven y con menos experiencia** termina generando el diseño real. Cayó en descrédito con razón.
- **Prototipado rápido y espiral**: en el fondo son **excusas para codificar antes** (empezar antes el ciclo build/test) y mantener a la **misma gente** en el alto nivel y en el código.

4.8 La “fantasía colectiva” y la conclusión

Existe la fantasía de que, si encontráramos **la notación gráfica correcta** para que los diseños de software se parecieran a los de otras ingenierías, ocuparíamos por fin nuestro lugar como ingenieros. Reeves **disiente**:

La ingeniería se trata de **cómo hacés el proceso**, no de si el producto final necesita un CAD para representarse.

El software es “**blando**”: puede representar cualquier cosa. Eso, sumado a la economía del ciclo build/test, hace **improbable** hallar métodos generales de validación más allá del ensayo y error. Pero **el proceso sí se puede mejorar**: si tratáramos el desarrollo como **un único proceso de diseño homogéneo** y mejoráramos sus fases más importantes (**programación, debug y test**), la industria sería más una disciplina rigurosa de lo que creemos.

Ideas clave del ensayo 3 (para fijar)

Analogía del puente: el diseño **no** es el puente → “¿cuál puente?”. Refinar el diseño antes de construir = ingeniería; en software se llama **testing y debugging**. **Economía:** el software es baratísimo de construir → preferimos **build/test** antes que pruebas formales. **Codificar es diseñar** (“los peces nadan”). **No** dice “olvidate del diseño”: pide reconocer que el diseño temprano **no está completo** y capturarlo en **un lenguaje de programación**. Las **notaciones independientes del lenguaje** agregan traducciones que **pierden información**. **Ingeniería = cómo hacés el proceso**, no si el resultado necesita un CAD.

5 Síntesis comparada de los tres ensayos

La tesis, en una frase

Programar es diseñar; el único diseño completo y no ambiguo es el código fuente; construir (compilar/linkear) es casi gratis y lo hacen las máquinas; validar y refinar el diseño es testing y debugging.

Qué aporta cada uno

Ensayo	Rol	Aporte distintivo
3 — Carta (1992)	Origen (más personal)	La analogía del puente (“¿cuál puente?”); la economía como explicación de todo; “los peces deben nadar”; las dos correcciones de percepción; crítica a notaciones independientes del lenguaje.
1 — “What Is Software Design?” (1992)	Desarrollo formal	El argumento del documento de diseño (criterio de manufactura); hardware vs. software; testing/debugging como diseño; crítica al proceso tradicional; rol de C++; documentación auxiliar; los 14 puntos finales.
2 — “13 Years Later” (2005)	Defensa / réplica	La distinción proceso vs. producto como eje; respuestas a las críticas A, B, B', C, D; “especificación = qué / diseño = cómo, interpretable mecánicamente”.

Conceptos transversales (aparecen en los tres)

- **Proceso vs. producto:** el malentendido raíz. “Código es diseño” no es “no diseños”.
- **Manufactura del software = compilar/linkear:** casi gratis, la hacen las máquinas.
- **Testing y debugging = validación y refinamiento del diseño**, no “control de calidad”.
- **Economía del build/test:** por qué no usamos pruebas formales.
- **Notaciones (UML, Booch, CASE, structure charts) no son el diseño;** el diseño termina en un lenguaje.
- **Documentación auxiliar:** necesaria, pero no es el diseño; el código manda.
- **Ingeniería = cómo hacés el proceso**, no si el resultado se ve “bonito” en un CAD.
- **No congelar** aspectos del diseño; todos interactúan y se refinan durante todo el ciclo.
- **C++ / mejores lenguajes** como el avance que de verdad mueve la aguja.

Conexión con la práctica moderna

Las tres piezas anticipan y fundamentan **Agile**, el **craftsmanship**, el **TDD/test-first** (el build/test como motor del diseño) y el **refactoring** (refinar el diseño —el código— de forma continua, sin “congelar” nada).